Service) on behalf of the user. Now we'll move from application authorization to user authorization. What can an authenticated user do in the system?

Spring Security associates each authenticated user with a list of `GrantedAuthority` objects that model the authorities the user has been granted. Granted authorities can be used to represent fine-grained permissions, roles, or even scopes and come from different sources depending on the authentication strategy. The authorities are available through the `Authentication` object representing the authenticated user and stored in the `SecurityContext`.

Since Catalog Service is configured as an OAuth2 Resource Server and uses JWT authentication, Spring Security extracts the list of scopes from the `scopes` claim of the Access Token and uses them as granted authorities for the given user automatically. Each `GrantedAuthority` object built in this way will be named with the `SCOPE_` prefix and the scope value.

The default behavior is acceptable in many scenarios where scopes are used to model permissions, but it doesn't fit our case where we rely on user roles to know which privileges each user has. We want to set up a role-based access control (RBAC) strategy using the user roles provided in the `roles` claim of the Access Token (see figure 12.6). In this section, I'll show you how to define a custom converter for the Access Token to build a list of `GrantedAuthority` objects using the values in the `roles` claim and the `ROLE_` prefix. Then we'll use those authorities to define authorization rules for the endpoints of Catalog Service.